

RTL STUDY · ARCHITECTURE · ROADMAP · SW LIBRARY

MIPI CSI-2 RX

RTL 스터디 & 설계 로드맵

D-PHY 수신 IP를 대상으로, PPI 경계부터 블록 단위로 분해하고
골든 모델을 축으로 단계적으로 진행하는 설계 계획

CSI-2

v2.0 · 프로토콜

D-PHY

v2.0 · 물리계층

PPI

RTL 설계 시작점

설계자

: 오동학 책임 연구원

: 김민지 연구원

: 서윤철 연구원

C-PHY 미포함 · D-PHY 단일 물리계층 기준

발표 구성

스펙 범위 → 데이터패스 블록 분해 → 단계별 로드맵 → 골든 모델 준비

01

배경과 스펙 범위

CSI-2 계층 구조와
처리 대상 데이터 타입

02

데이터패스 블록 분해

PPI부터 출력까지
블록 ① ~ ⑨ 개별 정리

03

설계 로드맵

Phase 0 ~ 5
산출물과 완료 기준

04

골든 모델 & 시뮬레이션 라이브러리

SW 참조 모델과 재사용 라이브러
리
검증 구조 전체 설계

이 문서에서 다루지 않는 것

— C-PHY 수신 경로

구현 범위에서 제외

— 출력 인터페이스 확정

ISP 연동 방식 미정

— 일정 · 공수 산정

블록 규모 확정 후 별도

01

배경과 스펙 범위

무엇을 만들 것인지 정하기 전에 어디까지 처리할지부터 정한다.

— CSI-2 / D-PHY 스펙 현황

— IP의 구현 범위

— 상세 스펙

대상 스펙과 구현 범위

최신 스펙 전체를 목표로 잡으면 첫 IP는 끝나지 않는다. 범위를 먼저 좁힌다.

MIPI CSI-2

v?

프로토콜 계층

- 패킷 헤더/푸터, VC, Data Type 정의
- 픽셀 언패킹 규칙을 규정
- 물리계층과 독립적으로 발전

MIPI D-PHY

v?..?

물리 계층

- 차동 페어 레인 + 전용 클럭 레인
- 표준채널 9 Gbps/lane
- 모든 데이터 레인이 클럭 공유

PPI

경계

RTL 시작점

- PHY와 프로토콜 계층의 디지털 경계
- PHY IP 구매 시 여기부터 자체 설계
- RxByteClkHS / RxDataHS / RxValidHS

IP 구현 범위

물리계층

D-PHY 단일
PPI Interface

레인 구성

1/2/3/4-lane 지원
De-shew

프로토콜

FS/FE/LS/LE + Long Packet

데이터타입

RAW · YUV · RGB

특수 기능(미정)

Scrambling, Compression

상세 스펙

D-PHY 기반 CSI-2 RX 단일 IP 기준. 미지원 항목은 이번 범위에서 의도적으로 제외한다.

지원 범위

■ PHY	MIPI D-PHY RX · PPI 인터페이스 기준, PHY는 외부 IP
■ Protocol	CSI-2 RX
■ Lane	1 / 2 / 3 / 4 Lane
■ Virtual Channel	VC0 · VC1 · VC2 · VC3 + VCX
■ Clock Mode	Continuous / Non-continuous
■ Packet	Short Packet / Long Packet
■ ECC	지원 — 1bit 정정 / 2bit 검출
■ CRC16	지원 — 검사 및 에러 리포트
■ Sync	FS / FE / LS / LE
■ Output	Pixel Stream (valid · line · frame sync), AXI4 Stream
■ Configuration	APB, AXI
■ Interrupt	지원 — 에러 및 프레임 이벤트

Data Type — 지원 목록

비이미지	Blanking Data Embedded 8-bit non-Image Data
YUV	YUV420 8-bit / 10-bit Legacy YUV420 8-bit YUV420 8-bit / 10-bit (Chroma Shifted Pixel Sampling) YUV422 8-bit / 10-bit
RGB	RGB444 / RGB555 / RGB565 / RGB666 / RGB888
RAW	RAW6 / RAW7 / RAW8 / RAW10 / RAW12 / RAW14
User Defined	User Defined 8-bit Data Type 1 ~ 4
압축	JPEG8

Embedded 8-bit non-Image Data는 Data Type 인식만 수행하고 페이로드는 drop 처리 (하단 참조)

미지원 — 이번 범위에서 제외

Compression	Embedded Data	Scrambling
—	Long Packet drop	—

02

데이터패스 블록 분해

PPI 경계에서 출력단까지, 블록 하나에 슬라이드 하나.

— CSI-2 계층 구조와 RTL 블록 대응

— 전체 데이터패스 개요

— 블록 ① ~ ⑨ 개별 정리

— 모듈 계층과 파라미터

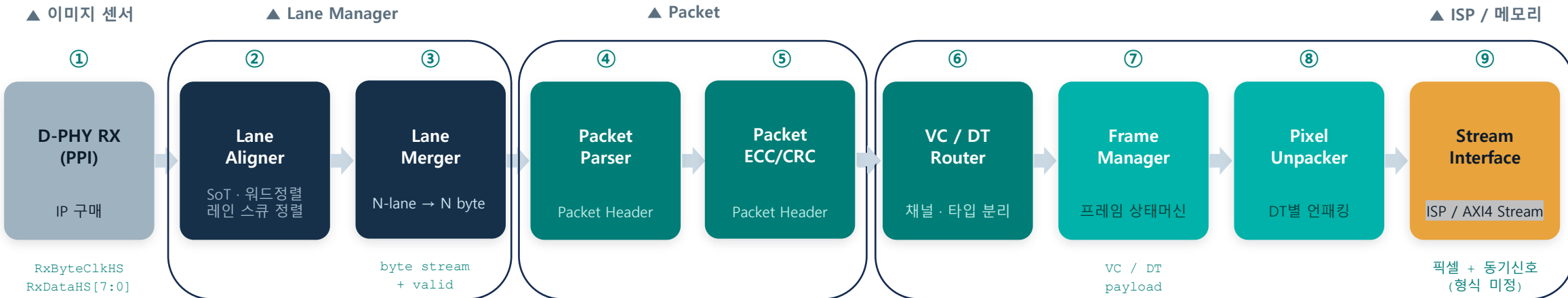
CSI-2 계층 구조와 RTL 블록 대응

각 계층이 곧 RTL 모듈 경계다. 계층 경계를 그대로 모듈 경계로 잡으면 검증 단위도 함께 나뉜다.



전체 데이터패스 개요

PHY IP는 구매하고 PPI 경계부터 자체 설계한다. ① ~ ⑨ 를 이어지는 슬라이드에서 하나씩 다룬다.



제어 · 상태 평면

CSR (APB, AXI)

레인 수, VC 매핑,
DT 필터, soft reset

Error Status

SoT · ECC · CRC,
WC 불일치, overflow

Debug Counter

프레임 · 라인 카운터,
성능 모니터

클럭 도메인

RxByteClkHS

bitrate / 8 · 센서 종속

pixel_clk

언패킹 이후 코어 도메인

xxx_clk

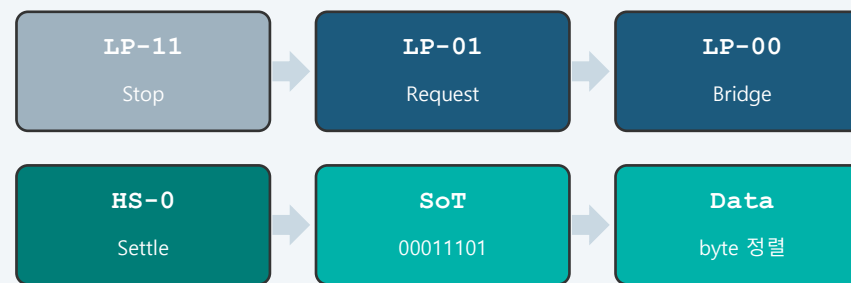
출력단 · 인터페이스 확정 후

PPI — PHY와 프로토콜 계층의 경계

이 신호 목록이 RTL 최상단 포트 사양이 된다.

RxByteClkHS	out	HS 수신 바이트 클럭 (bitrate / 8)
RxDataHS[7:0]	out	레인별 수신 바이트
RxValidHS	out	데이터 유효 스트로브
RxActiveHS	out	HS 수신 중 · 하강 시 EoT
RxSyncHS	out	SoT 검출 완료 · 바이트 경계 확정
Stopstate	out	레인 LP-11 정지 · 초기화 완료 판정
Enable	in	레인 활성화
ForceRxmode	in	강제 수신 모드 진입
ErrSotHS	out	SoT 에러 · 정정 가능
ErrSotSyncHS	out	SoT 동기 실패 · 전송 폐기
ErrControl	out	LP 제어 시퀀스 위반

LP → HS 전환 시퀀스



RxActiveHS 상승 후 RxSyncHS 펄스 시점에 바이트 경계가 확정된다.

핵심 결정 사항

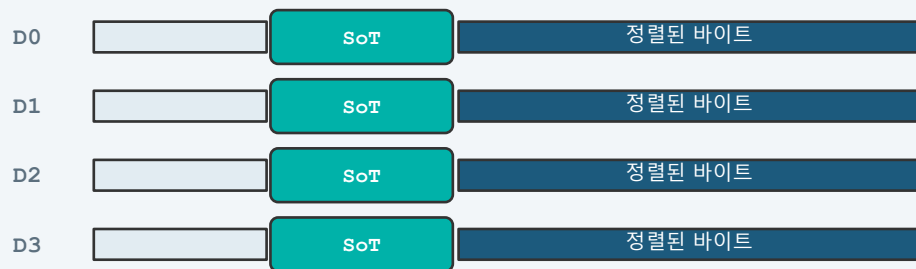
- PPI 데이터 폭 — 8b/lane 유지 또는 기어링 적용
- PHY가 레인 정렬을 어디까지 해주는지 먼저 확인
- SoT 에러 두 종류의 처리 정책 구분

Lane Aligner & Align Buffer

두 블록은 역할이 이어져 있어 함께 설명한다. ②는 각 레인의 바이트 경계를 찾고, ③은 레인 간 도착 시점을 맞춘다.

② Lane Aligner — 바이트 경계 찾기

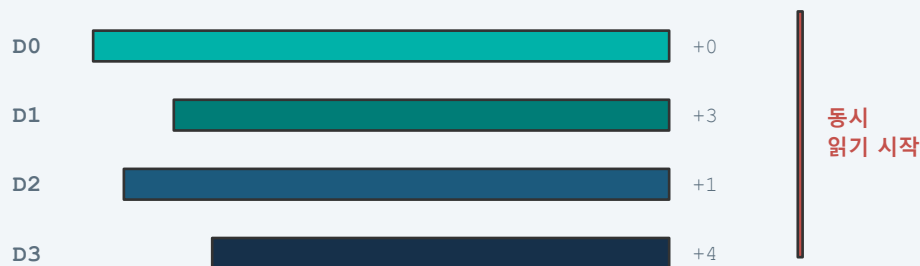
레인마다 독립적으로 SoT 패턴을 검출해, 어느 비트부터가 바이트의 시작인지 확인한다.



출력 — 레인별 정렬 완료 플래그 + 정렬된 바이트

③ Align Buffer — 레인 간 시점 맞추기

레인마다 SoT 검출 시점이 다르다. 전 레인이 준비될 때까지 기다렸다가 동시에 읽기 시작한다.



깊이 = 최대 허용 레인 간 스큐(byte 단위) + 여유 1~2 — 보통 4면 충분

1 비동기 FIFO가 아니다

D-PHY는 전용 클럭 레인이 있어 전 레인이 같은 byte clock을 공유한다. 주파수 차이가 없으므로 얇은 동기 정렬 버퍼면 충분하다.

2 비트 스큐는 PHY 담당

클럭과 데이터 사이의 비트 레벨 스큐는 PHY 내부 딜레이 라인이 잡는다. 1.5 Gbps 초과 시 deskew burst로 캘리브레이션된다.

3 실패 모드

정렬 실패 시 프레임 전체가 조용히 깨진다. 버퍼 깊이를 넘는 스큐는 에러로 검출하고, 레인별 지연값을 디버그 레지스터로 노출한다.

Lane Merger — 단일 바이트 스트림 복원

송신 측이 바이트를 레인에 순서대로 뿌린 것을, 같은 규칙으로 되돌린다.

4-lane 라운드로빈 분배 → 병합



병합 결과 — 단일 바이트 스트림



B0 → B1 → B2 → ... 순서로 복원

레인 수가 2이면 D0/D1만, 1이면 D0만 사용 — 병합기는 모듈러 카운터의 계수만 바뀐다.

1 출력 폭

1 byte/clock로 두면 가장 단순하지만 후단 클럭이 높아진다. 레인 수 × 1 byte로 내보내면 병합기 자체가 거의 필요 없다.

2 패킷 경계

패킷의 시작과 끝은 병합된 스트림 기준으로 판정한다. 패킷 간 간격에서는 valid를 내리고, EoT 시점에 잔여 바이트를 정리한다.

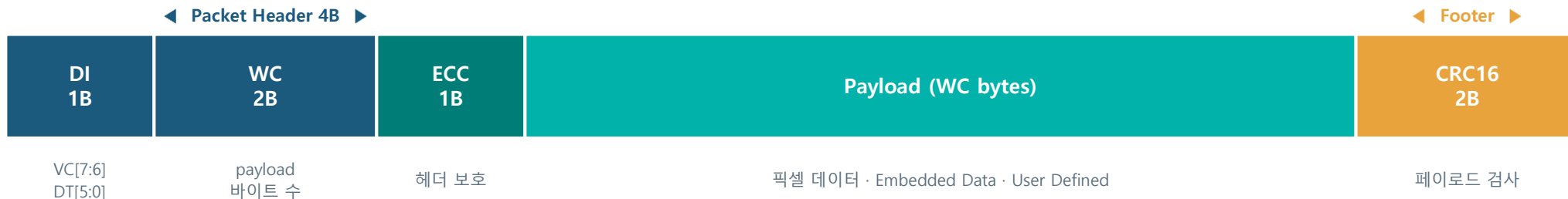
3 레인 수 변경

활성 레인 마스크가 바뀌는 순간 잔여 데이터가 남으면 첫 프레임이 깨진다. 변경은 프레임 경계에서만 허용한다.

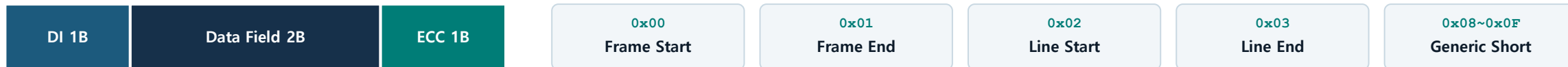
Packet Parser — 패킷과 프레임 구조

바이트 스트림을 패킷 단위로 끊어 읽는다. CSI-2의 실질적 핵심.

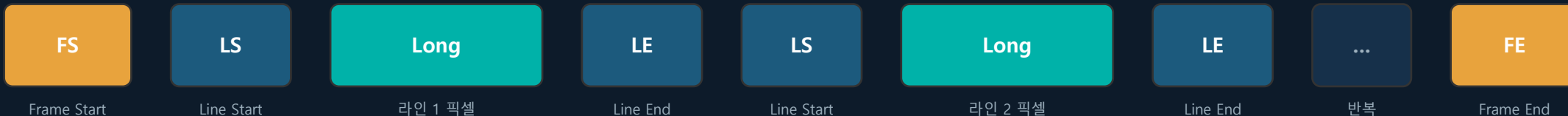
Long Packet — 픽셀 데이터를 나르는 패킷



Short Packet — 동기 신호용, 페이로드 없음



한 프레임의 패킷 시퀀스



LS / LE는 선택 사항이다. 센서가 생략하면 Long Packet 개수로 라인을 세야 하므로, 두 경우를 모두 처리하도록 설계한다.

Packet Parser — ECC와 CRC

헤더는 ECC로 보호하고 정정까지, 페이로드는 CRC로 검사만 한다.

ECC — 헤더 보호 (Hamming 부호)

보호 대상 24비트 (DI 8b + WC 16b)

패리티 6b

- **에러 없음** 그대로 통과
- **1비트 에러** 정정 후 진행 · 카운터 증가
- **2비트 이상** 정정 불가 · 패킷 폐기

VCX 사용 시 보호 대상은 26비트. VCX 미지원 송신기에서 온 헤더는 비트 24-25를 0으로 강제한 뒤 신드롬을 계산해야 한다 (VCX Override).

CRC16 — 페이로드 검사

- 생성 다항식 $x^{16} + x^{12} + x^5 + 1$
- 초기값 0xFFFF
- 연산 범위 페이로드 바이트 전체 (헤더 제외)
- 구현 테이블 대신 병렬 XOR 트리
- 불일치 시 정정 불가 — 카운터 증가 후 진행

```
#define POLY 0x8408 /* 1021H bit reversed */
unsigned short crc16(char *data_p, unsigned short length)
{
    unsigned char i;
    unsigned int data;
    unsigned int crc = 0xffff;
    if (length == 0)
        return (unsigned short) (crc);
    do
    {
        for (i=0; data=(unsigned int)0xff & *data_p++;
             i < 8; i++, data >>= 1)
        {
            if ((crc & 0x0001) ^ (data & 0x0001))
                crc = (crc >> 1) ^ POLY;
            else
                crc >>= 1;
        }
        while (--length);
    } while (1);
    // Uncomment to change from little to big Endian
    // crc = (crc & 0xff) << 8 | ((crc & 0xff00) >> 8);
    return (unsigned short) (crc);
}
```

Figure 69 16-bit CRC Software Implementation Example

WC와 실제 길이가 다르면 CRC 이전에 이미 스트림이 어긋난다 — 별개 에러로 분류.

에러 처리 정책 — 코딩 전에 표로 합의할 것

SoT 에러

카운터만

수신 계속

SoT 동기 실패

전송 폐기

다음 SoT 대기

ECC 1비트

정정 후 진행

카운터 증가

ECC 정정 불가

패킷 폐기

다음 헤더 탐색

CRC 불일치

페이로드 통과

프레임에 플래그

WC 불일치

패킷 폐기

다음 FS까지 대기

출력단 정체

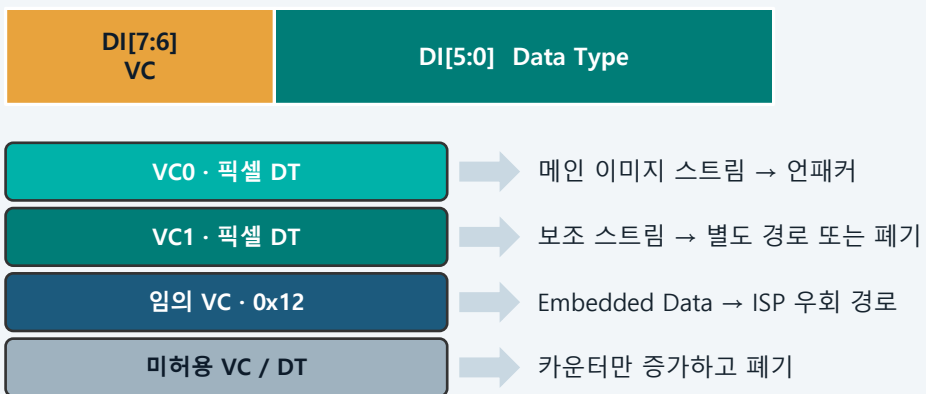
프레임 폐기

다음 FS에서 재동기

VC / DT Router — 패킷 분류와 필터

디코드된 패킷을 어디로 보낼지만 결정한다. 프레임 상태 관리는 다음 블록의 역할.

DI 1바이트가 분류의 전부



CSR로 노출할 필터

<code>vc_enable[3:0]</code>	수신할 Virtual Channel 마스크
<code>dt_enable[...]</code>	허용 Data Type 화이트리스트
<code>embedded_route</code>	Embedded Data 출력 경로 선택
<code>drop_count</code>	폐기된 패킷 수 (읽기 시 클리어)

1 조기 폐기가 목적

관심 없는 채널·타입을 여기서 버리면 언패커와 출력단의 부담이 줄어든다. 필터는 데이터패스에서 가장싼 위치에 둔다.

2 Embedded는 픽셀이 아니다

0x12는 센서가 붙이는 메타데이터다. 이미지 경로에 섞이면 ISP가 오동작하므로 반드시 분리한다.

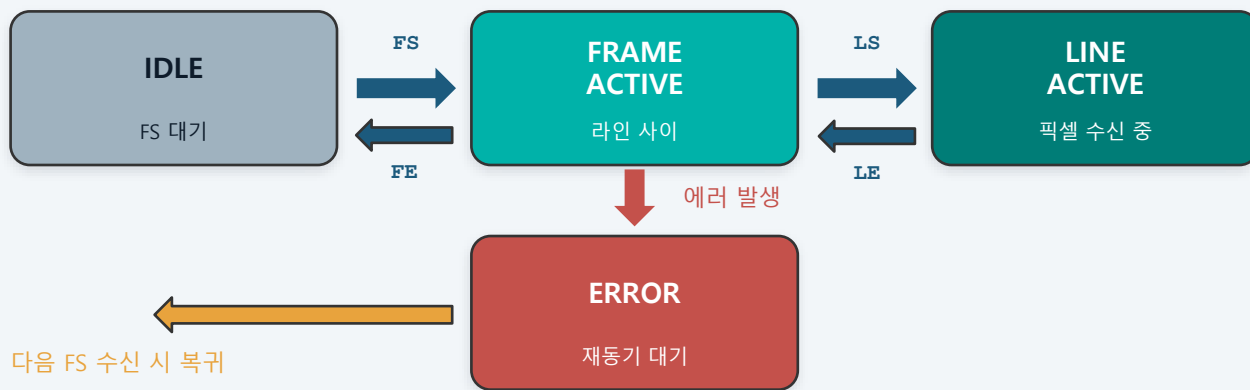
3 VCX는 CSI-2 V2에서 추가

확장 VC(최대 32채널)는 헤더 형식이 달라진다.

Frame Manager — 프레임 상태머신

패킷의 순서가 맞는지 감시하고, 에러가 나면 어디서 다시 시작할지 결정한다.

상태 천이



IDLE 복귀 조건은 FE 하나로 두고, 에러 상태에서는 FS만 인정한다 — 재동기 지점을 하나로 고정해야 복구 루프에 빠지지 않는다.

감시 항목

프레임 번호

FS/FE 데이터 필드의 연속성 → 프레임 누락 검출

라인 카운트

LE 수신 또는 Long Packet 완료 시 증가, FE에서 대조

순서 위반

FS 없이 데이터 도착, FE 없이 다음 FS 등

라인 길이

WC가 프레임 내내 일정한지 확인

1 상태 천이표를 먼저

코딩 전에 상태 × 이벤트 표를 만들고 리뷰한다. 이 표가 그대로 검증 시나리오 목록이 된다.

2 프레임 단위로 판정

에러가 나면 라인 단위로 살리려 하지 말고 프레임 전체를 버리는 편이 안전하다.

3 상태를 밖으로 노출

현재 상태와 마지막 에러 원인을 CSR로 읽을 수 있게 하면 브링업 시간이 크게 줄어든다.

Pixel Unpacker — 지원 Data Type

언패커는 DT별 서브모듈의 집합이다. 지원 목록이 곧 IP의 사양이 된다.

비이미지 데이터	
0x00	Frame Start Code
0x01	Frame End Code
0x02	Line Start Code (Optional)
0x03	Line End Code (Optional)
0x10	Null
0x11	Blanking Data
0x12	Embedded 8-bit non-Image Data

YUV	
0x18	YUV420 8-bit
0x19	YUV420 10-bit
0x1A	Legacy YUV420 8-bit
0x1C	YUV420 8-bit (Chroma Shifted)
0x1D	YUV420 10-bit (Chroma Shifted)
0x1E	YUV422 8-bit
0x1F	YUV422 10-bit

RGB	
0x20	RGB444
0x21	RGB555
0x22	RGB565
0x23	RGB666
0x24	RGB888

RAW	
0x28	RAW6
0x29	RAW7
0x2A	RAW8
0x2B	RAW10
0x2C	RAW12
0x2D	RAW14
0x2E~0x2F	RAW16 / RAW20 (v2.0+ 확장)

User Defined	
0x30	User Defined 8-bit Data Type 1
0x31	User Defined 8-bit Data Type 2
0x32	User Defined 8-bit Data Type 3
0x33	User Defined 8-bit Data Type 4

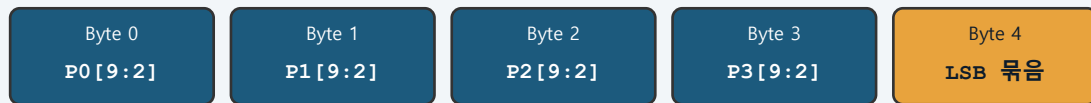
압축	
—	JPEG8
	별도 DT 코드가 아니라 User Defined Type 1로 전송하는 것이 관례

각 타입은 독립 서브모듈로 두고 DT 디코드 결과로 먹스한다 — 지원 타입 추가가 국소 변경으로 끝난다.

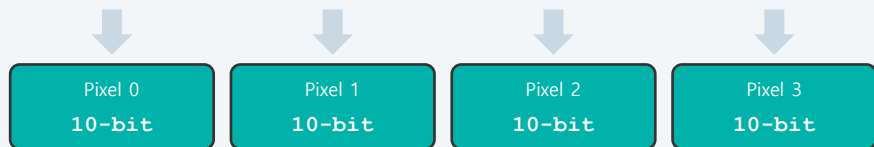
Pixel Unpacker — 비트 매핑

타입마다 바이트와 픽셀의 대응이 다르다. 오프바이원 버그가 가장 많이 나오는 지점.

RAW10 — 5바이트 = 4픽셀

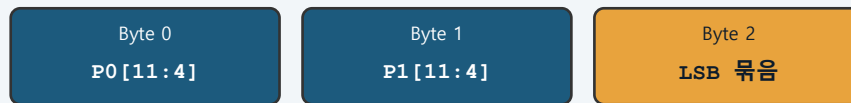


Byte4 = P3[1:0] | P2[1:0] | P1[1:0] | P0[1:0]

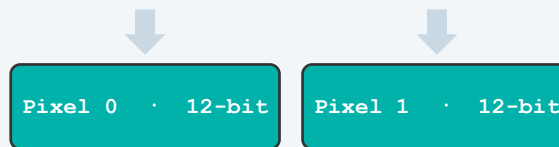


라인 길이가 4의 배수가 아니면 마지막 그룹이 부분 채움 — WC 기준으로 잔여 처리 필수

RAW12 — 3바이트 = 2픽셀



Byte2 = P1[3:0] | P0[3:0]



구조는 RAW10과 동일 — MSB 바이트들 뒤에 LSB를 모아 붙인다

타입별 바이트 / 픽셀 비율

RAW6 3B = 4px	RAW7 7B = 8px	RAW8 1B = 1px	RAW10 5B = 4px	RAW12 3B = 2px	RAW14 7B = 4px
YUV422 8-bit 4B = 2px	YUV422 10-bit 5B = 2px	RGB444/555/565 2B = 1px	RGB666 9B = 4px	RGB888 3B = 1px	Embedded · JPEG8 바이트 그대로

YUV420 계열은 짝수/홀수 라인의 구성이 달라 라인 패리티를 상태로 유지해야 한다.

출력 인터페이스 — 미확정 영역

ISP 연동 방식이 정해지지 않았다. 인터페이스와 무관하게 지금 확정할 수 있는 것부터 정리한다.



출력 인터페이스 후보

AXI4-Stream 범용 · IP 재사용 쉬움 tuser로 SOF, tlast로 EOL 전달	벤더 비디오 스트림 ISP와 바로 연결 벤더 규약에 종속 · 이식성 낮음	단순 픽셀 버스 valid + 동기신호만 구현 최소 · 백프레셔 없음	DMA 라인 라이트 메모리에 직접 기록 라인버퍼와 주소 생성기 필요
---	---	---	--

인터페이스가 정해지기 전에도 확정할 것	— SOF · EOL 표시 방법	— 백프레셔 유무	— 정체 시 정책
	형식은 달라도 프레임/라인 경계 신호는 반드시 필요	받을 수 있는지에 따라 라인버퍼 필요 여부 가 갈린다	센서는 못 멈추므로 프레임 폐기 규칙을 미리 정한다

모듈 계층과 파라미터

블록 분해가 끝나면 그대로 파일 구조가 된다.

모듈 계층

csi2_rx_top

└ csi2_rx_ppi_if	CSI-2 RX 전체 모듈 통합 및 모듈 간 연결
└ csi2_rx_lane_mgr	D-PHY PPI 신호 수신 및 내부 인터페이스 정규화
└ csi2_rx_lane × N	Multi-Lane 전체 관리, Lane 동기화/정렬 및 병합
└ csi2_rx_lane_merge	Lane 별 데이터 수신, SoT 검출, Lane 상태 및 De-Skew 처리
└ csi2_rx_pkt	N개의 Lane의 Byte를 CSI-2 순서에 맞게 하나의 데이터로 병합
└ csi2_rx_pkt_parser	CSI-2 Packet Layer 전체 제어 및 Packet 처리
└ csi2_rx_ecc	SoT/EoT 검출, Header 파싱 및 Packet Type/VC/DC/WC 해석
└ csi2_rx_crc16	CSI-2 Header ECC 계산 및 오류 검출/정정
└ csi2_rx_vc_router	Long Packet Payload CRC-16 계산 및 오류 검출
└ csi2_rx_frame_mgr	VC/DT 기반 Packet 필터링 및 데이터 경로 분배
└ csi2_rx_unpack	Frame/Line 상태 관리, FS/FE/LS/LE 처리 및 Frame/Line 정보
└ csi2_rx_stream_if	CSI-2 Payload를 RAW / YUV / RGB 등 Pixel format으로 변환
└ csi2_rx_csr	변환된 Pixel/Byte Stream을 외부 ISP/DMA 인터페이스로 출력
	Configuration register, Status/Err register, Interrupt 제어

주요 파라미터

LANE_NUM = 4	지원 D-PHY 데이터 레인 수 (1~4)
PPI_WIDTH = 8	Lane 당 PPI 데이터 비트 폭 (8,16)
VC_WIDTH = 2	Virtual Channel 식별 비트 폭
DT_WIDTH = 6	CSI-2 Data Type 필드 비트 폭
MAX_PIXEL_WIDTH = 14	최대 지원 픽셀 비트 폭 (RAW14 기준)
AFIFO_DEPTH = 16	Lane 간 비동기 FIFO 깊이

03

설계 로드맵

스펙 독해에서 보완 검증까지,
산출물 단위로 끊어 간다.

 Phase 0 ~ 5 진행 경로

 단계별 산출물

 완료 기준

Phase 0 ~ 5

각 Phase는 '읽었다'가 아니라 '동작하는 산출물'로만 종료를 선언한다. 소요 기간은 별도 산정.



M1 Phase 1 종료

모델이 실제 센서 덤프를 이미지로 복원

M2 Phase 2 종료

모든 모듈이 단위 TB를 통과

M3 Phase 3 종료

통합 RTL이 프레임용 끝단까지 전달

M4 Phase 4 종료

목표 주파수에서 타이밍 클로저

단계별 산출물과 완료 기준

완료 기준이 없으면 Phase는 끝나지 않는다. 측정 가능한 문장으로만 적는다.

단계	주제	필요 역량	산출물	완료 기준
Phase 0	스펙 독해	문서 독해 · 프로토콜 이해	PPI 포트 사양서, 패킷 필드 표	타깃 센서의 프레임 구조를 손으로 그릴 수 있다
Phase 1	골든 모델 · 시뮬 환경	Python · 비트 조작 · TB 구성	자극 생성기, 참조 모델, TB 골격	모델이 실제 센서 덤프를 정상 이미지로 복원한다
Phase 2	개별 모듈 RTL	SystemVerilog · FSM · 파이프라인	모듈별 RTL + 단위 TB	모든 모듈이 단위 TB에서 참조 모델과 일치한다
Phase 3	통합 모듈 설계	인터페이스 정합 · CDC · 에러 처리	통합 RTL, CSR 맵, 블록 사양서	프레임이 입력단부터 출력단까지 무손실 전달된다
Phase 4	최적화 및 타이밍	합성 · STA · 저전력	합성 스크립트, 타이밍 리포트	목표 주파수에서 셋업·홀드 위반 0건
Phase 5	보완 검증	코너 케이스 · 회귀 · 스크립팅	검증 리포트, 회귀 스위트	에러 시나리오 목록 전량 통과, 미결 이슈 0건

참고 자료

MIPI 스펙 원문(회원 전용) · Xilinx PG232 / PG202 · Microchip PolarFire CSI-2 RX 유저가이드 · Linux drivers/media 소스 · FPGA 벤더 레퍼런스 디자인

04

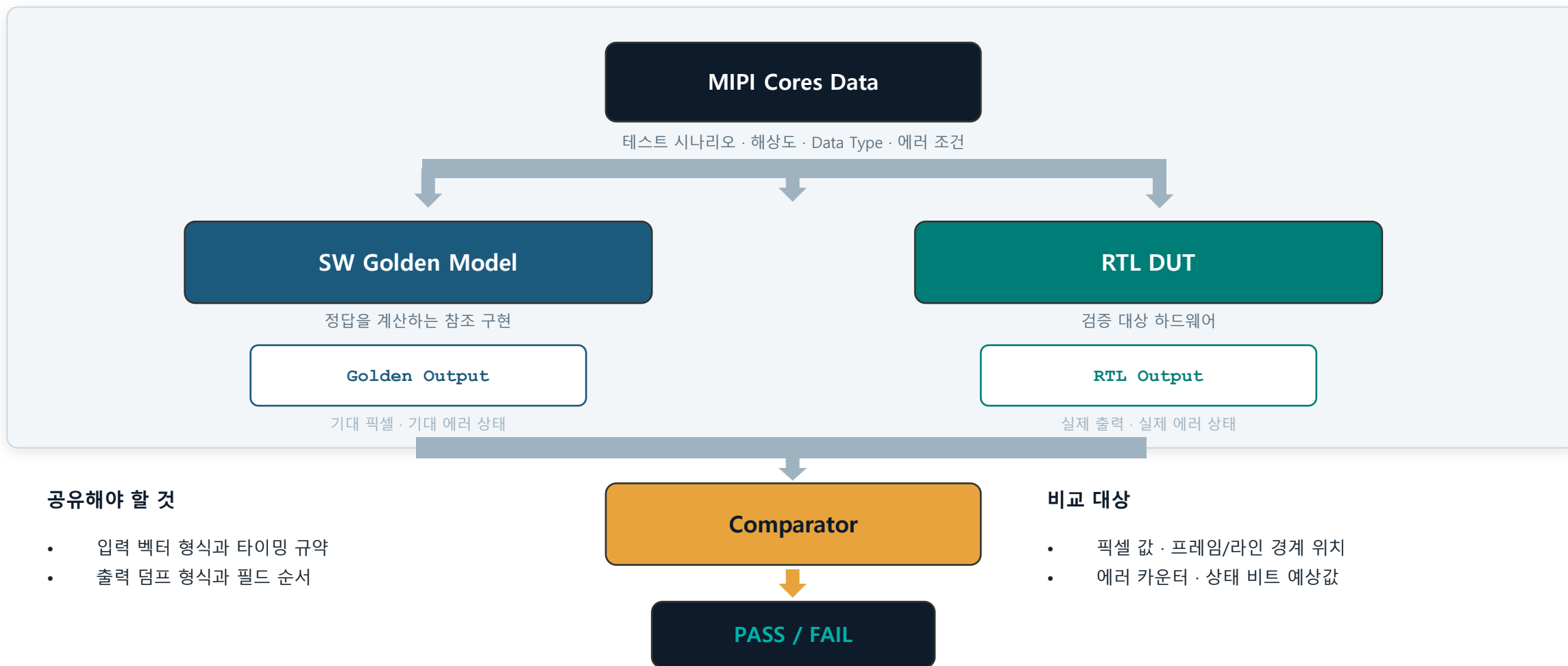
골든 모델 & 시뮬레이션 라이브러리

RTL을 검증하려면 정답이 먼저 있어야 한다.
라이브러리를 만들고, 그것으로 정답을 조립한다.

- 검증 구조와 개발 순서
- 라이브러리 구성과 모듈 상세
- 최종 SW 아키텍처

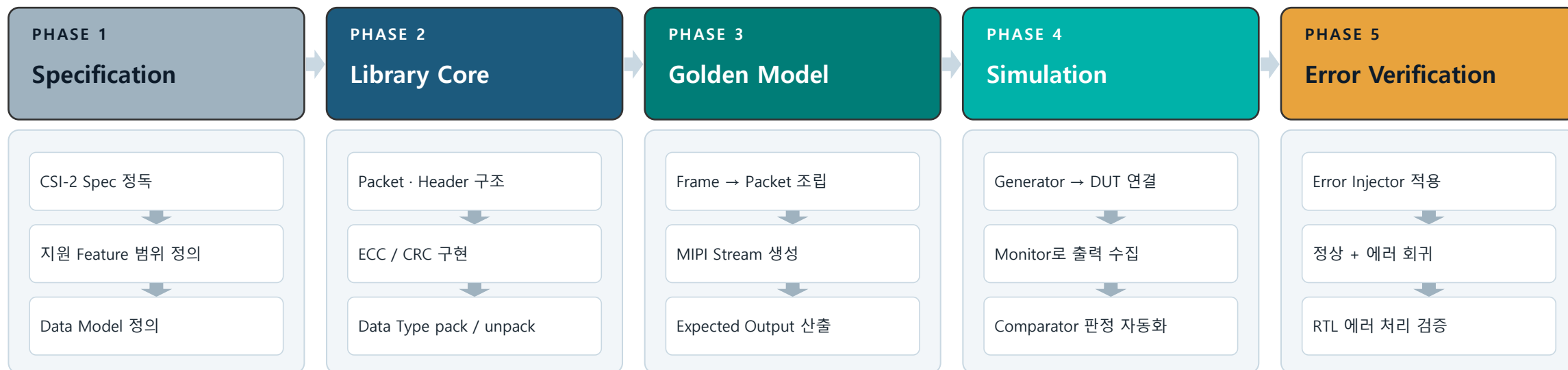
검증 구조 — 같은 입력, 두 개의 출력

RTL과 SW 모델이 처음부터 동일한 입력·출력 정의를 공유하도록 구조를 잡는 것이 출발점이다.



라이브러리 개발 순서

챕터 3의 RTL 로드맵과 병행한다. 라이브러리 Phase 1~3이 RTL Phase 1에 해당한다.



챕터 3 RTL 로드맵과의 대응

Library Phase 1~3

→ RTL Phase 1

골든 모델 및 시뮬레이션 환경 준비

Library Phase 4

→ RTL Phase 2~3

개별 모듈이 나올 때마다 병행 적용

Library Phase 5

→ RTL Phase 5

보완 검증 단계의 에러 시나리오 공급

MIPI Library (core) vs Golden Model

라이브러리는 프로토콜 자체를 구현한 부품, 골든 모델은 그 부품으로 RTL과 동일한 동작을 재현한 참조 구현.

MIPI Library (core)		재사용 부품	Golden Model		RTL과 동작 동일
성격	MIPI 프로토콜의 SW 구현		성격	RTL이 내야 할 출력을 계산	
의존	프로젝트에 종속되지 않음		의존	라이브러리를 호출해 조립	
알고리즘	여기에만 존재 (ECC · CRC · 패킹)		알고리즘	없음 — 순서와 설정만 담당	
수명	다음 MIPI 프로젝트에서 재사용		수명	이 IP 프로젝트와 함께	
테스트	모듈 단독 · 왕복 · 전수 검증		테스트	실 센서 덤프 · 왕복 일치	

“RTL과 동작 동일” — 블록 단위로 1:1 대응시킨다			
RTL 블록		Golden Model 단계	
④ Lane Merger		레인 분배 · 병합	
⑤ Packet Decoder		헤더 파싱 · ECC · CRC	
⑧ Pixel Unpacker		Data Type별 언패킹	

MIPI 시뮬레이션 라이브러리 구조

재사용 라이브러리(py + rtl)와 프로젝트 전용 영역을 분리한다

핵심 포인트

- ✓ core는 ecc · crc16 · packet · packing 네 개로 시작
- ✓ 프로토콜 지식은 py, 타이밍 재생은 rtl
- ✓ MIPI/ 는 재사용 자산, ISP_MIPI/ 는 프로젝트 종속
- ✓ 벡터 포맷이 py ↔ rtl 사이의 유일한 계약

폴더 구조

```
SIM/LIBRARY/MIPI/
├─ py/
│   └─ core/
│       ├── ecc.py
│       ├── crc16.py
│       ├── packet.py
│       └── packing.py
├─ rtl/
│   ├── mipi_ppi_gen.sv
│   ├── mipi_ppi_mon.sv
│   └── mipi_out_dump.sv
├─ DOC/ vector_format.md
└─ VERSION
```

재사용

```
SIM/Func/ISP_MIPI/
├─ DOC/
│   ├── block_diagram
│   ├── csr_map
│   └── *.md
├─ SW/
│   ├── golden.py
│   ├── vector.py
│   └── check.py
├─ TEST_SET/
│   ├── *.bmp
│   ├── *.raw
│   └── *.png
└─ SIM/
    ├── SIM_MIPI_ECC/
    ├── SIM_MIPI_CRC16/
    ├── SIM_MIPI_UNPACK/
    └── SIM_MIPI_TOP/
```

프로젝트

입력 이미지

모듈 역할

모듈	역할	주요 내용
<code>ecc.py</code>	헤더 무결성	패리티 생성 · 신드롬 · 1bit 정정 / 2bit 검출 · VCX override
<code>crc16.py</code>	페이로드 무결성	CRC16 생성 · 검사, 병렬 N바이트 버전
<code>packet.py</code>	패킷 조립 · 해석	Short / Long, DI · WC · ECC · payload · CRC, 재동기
<code>packing.py</code>	픽셀 ↔ 바이트	Data Type별 pack / unpack, 잔여 처리, WC 산출
<code>rtl/</code>	벡터 재생	벡터 → PPI 파형, 관찰 · 덤프 — 프로토콜 지식 없음

전체 흐름



RTL 생성기는 프로토콜을 모른다

벡터 재생만 담당

자극과 정답은 한 함수에 서

따로 만들면 어긋난다

생성기는 고정, 벡터만 교체

포맷 확정 → 생성기 → 벡터 순

포맷은 문서로 계약

vector_format.md

용어

Packet : CSI-2 패킷 (DI · WC · ECC · Payload · CRC) · PPI : PHY와 프로토콜 계층의 경계 신호 · Golden Model : 정답 참조 모델(SW) · DUT : 검증 대상 RTL